CSE4203 — Principles of Software Architecture

University Appointment & Student Services System (UASSS)

Milestone 1 — Proposal & Architectural Requirements

Instructor: Dr. Nada Mahmoud Hassan

TA: Eng. Mohamed Fareh

Submission date: October 23, 2026

Team name: [SoloQ]

Member names and IDs: Ahmed Adel - 221004576.

1. System Overview

The University Appointment & Student Services System (UASSS) is a proposed web-based platform that helps university students arrange appointments with university departments. Students can view available time slots, book or cancel appointments, and receive email confirmations. Department staff can manage their availability and review upcoming appointments. Administrators can manage department information and staff accounts.

The system addresses problems that can occur with manual appointment scheduling, including conflicting bookings, delayed confirmations, and difficulty viewing availability. Its main users are students, department staff, and administrators. The system will interact with an external email notification service to send booking confirmations and, if included in a later version, reminders.

From an architecture perspective, the system must protect student information while remaining easy to use. It must also prevent two students from booking the same slot at the same time. A separated web frontend and backend API, supported by a persistent database, is a reasonable initial design direction. The email integration should be separated from core booking logic so that the notification provider can be changed without rewriting appointment rules.

The Milestone 1 proposal focuses on requirements and architecture drivers. A later prototype is expected to implement a small, testable subset: authentication, viewing available slots, booking and cancelling appointments, staff schedule management, prevention of conflicting bookings, and test email confirmations. These are proposed scope decisions and should be confirmed with the instructor/team.

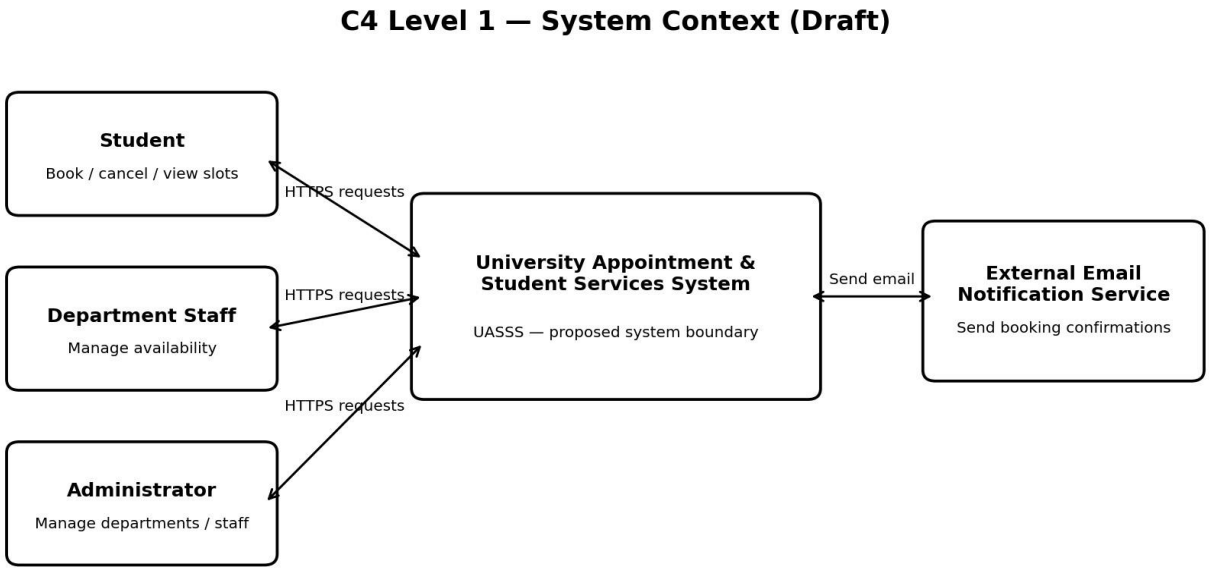
2. Stakeholders and Concerns

Stakeholder	Key concerns
Students	Easy booking and cancellation; accurate availability; privacy; timely confirmation.
Department staff	Ability to manage schedules; accurate appointment lists; fewer conflicts.
System administrators	Manage departments and accounts; enforce permissions; investigate issues.
University management	Efficient service; reliability; useful operational information.
Development team	Clear requirements; testability; maintainability; manageable scope.
IT support / operations	Monitoring; backup and recovery; secure configuration; incident handling.
External email provider	Valid requests; delivery limits; credentials; provider availability.

3. System Context (C4 Level 1)

The context diagram represents UASSS as one system. It shows the people and external service that interact with it. Internal implementation components such as the frontend, backend, and database should not be drawn as separate systems in this Level 1 context diagram.

Diagram relationships: Student uses UASSS to view slots, book and cancel appointments; Department Staff uses UASSS to manage availability and view appointments; Administrator uses UASSS to manage departments and staff; UASSS sends confirmation emails through the external Email Notification Service.



Legend: boxes = people / systems; arrows = interactions. System is shown as one box.

4. Functional Requirements (MoSCoW)

ID	User story / requirement	Priority	M3 prototype?
FR1	As a student or staff member, I can sign in so that access is associated with a user account.	Must	Yes
FR2	As a student, I can view departments and available appointment slots.	Must	Yes
FR3	As a student, I can book an available appointment slot.	Must	Yes
FR4	As a student, I can cancel my own appointment.	Must	Yes
FR5	As a student, I receive an email confirmation after a successful booking.	Must	Yes (test provider)
FR6	As department staff, I can view upcoming appointments for my department.	Must	Yes
FR7	As department staff, I can create and update available time slots.	Must	Yes
FR8	The system prevents two active bookings for the same slot.	Must	Yes
FR9	As an administrator, I can create and update department records.	Should	No
FR10	As a student, I can receive a reminder before an appointment.	Should	No
ID	User story / requirement	Priority	M3 prototype?
FR11	As an administrator, I can view basic appointment statistics.	Could	No
FR12	As a student, I can reschedule an existing appointment.	Could	No

MoSCoW meaning: Must = essential for the proposed prototype; Should = important but not essential for the first prototype; Could = desirable if time permits; Won't (this time) = deliberately excluded from the current prototype. The table above does not use a Won't item because the 12 listed requirements are prioritized as Must, Should, or Could.

5. Quality Attribute Scenarios

Each scenario uses the six-part form required in the assignment: source, stimulus, artifact, environment, response, and response measure.

QA-1 — Performance

Source: Student. Stimulus: requests available appointment slots. Artifact: availability API. Environment: normal operation with 200 concurrent users. Response: the service retrieves and returns available slots. Response measure: 95% of availability requests complete within 500 ms, measured at the service boundary and excluding external network latency.

QA-2 — Security

Source: user without permission. Stimulus: attempts to view or modify another student's appointment. Artifact: authentication and authorization layer. Environment: normal operation. Response: the request is rejected and protected data is not returned. Response measure: 100% of defined unauthorized-access test cases are rejected, with no protected appointment data exposed.

QA-3 — Availability / Recoverability

Source: student. Stimulus: attempts to use booking after a recoverable backend interruption. Artifact: appointment backend. Environment: the failed component has been restored. Response: the service resumes accepting requests and maintains booking integrity. Response measure: service resumes within 5 minutes after restoration, and recovery tests produce no duplicate active bookings.

QA-4 — Consistency / Data integrity

Source: two students. Stimulus: both submit a booking for the same slot nearly simultaneously. Artifact: booking service and database. Environment: concurrent requests. Response: one booking succeeds and the conflicting request is rejected. Response measure: exactly one active booking exists for the slot in 100 repeated concurrent-booking test runs.

QA-5 — Modifiability

Source: developer. Stimulus: replaces or adds an email notification provider. Artifact: notification integration component. Environment: development and test environment. Response: the provider can be changed without modifying the core appointment rules. Response measure: a developer completes the change within 4 hours and all existing booking tests pass.

QA-6 — Usability

Source: first-time student user. Stimulus: attempts to book an available appointment. Artifact: student-facing web interface. Environment: desktop browser, no prior training. Response: the user locates a slot and completes the booking. Response measure: at least 9 of 10 representative test participants complete the task within 2 minutes without assistance.

6. Constraints

Type	Constraint / assumption	Status
Technical	System is accessible through a modern web browser.	Proposed requirement
Technical	Frontend and backend API are clearly separated.	Proposed architectural decision
Technical	Appointment records are stored persistently.	Proposed requirement
Technical	Email confirmation uses an external email service or test provider.	Proposed integration
Business	Initial scope is appointment scheduling, not a complete university management system.	Proposed scope
Organizational	The assignment brief says each group must have exactly five students.	Explicit course constraint — individual submission needs instructor approval

Organizational	All project work is submitted through GitHub.	Explicit course constraint
Schedule	Milestone 1 due date is October 23, 2026.	Explicit course constraint

Confirm whether the course requires a particular programming language, database, deployment platform, or budget. These are not specified in the provided Milestone 1 brief, so do not claim that a particular technology is professor-mandated unless you have separate instructions.

7. Architecturally Significant Requirements (ASRs)

Rank	Requirement	Business value	Architectural impact	Why it matters
1	QA-2 Security and access control	High	High	Influences authentication, role-based authorization, API protection, and data access.
2	QA-4 / FR8 Booking consistency	High	High	Requires transactional booking logic and a database constraint or equivalent concurrency protection.
3	QA-1 Performance	High	High	Influences query design, indexing, response-time measurement, and load testing.
4	QA-3 Availability / recovery	High	Medium	Influences error handling, monitoring, backup, and recovery procedures.
5	QA-5 Modifiability	Medium	High	Supports separating the notification integration from appointment business rules.

Architecturally Significant Requirements (ASRs) — Prioritization Summary

After reviewing the main requirements of the University Appointment & Student Services System, I ranked security, booking consistency, and performance as the highest-priority architecturally significant requirements. I chose these three because the system needs to protect students' personal information, prevent multiple students from booking the same appointment slot, and respond to requests within an acceptable time. I also considered availability and modifiability important, since the system should recover from service interruptions and allow the email notification service to be changed without significantly affecting the main booking functionality.

For the utility tree, I identified the main business goal as providing a reliable and secure appointment service for students and university staff. I organized the quality attributes under this goal into security, data integrity, performance, availability, and modifiability. Based on my assessment of their importance to the system, I gave the highest priority to QA-2 (Security), QA-4 (Booking Consistency), and QA-1 (Performance). This prioritization is based on the potential effect each requirement could have on the correct and reliable operation of the system.

Reference

Software Engineering Institute, Carnegie Mellon University, “Quality Attribute Workshop Collection.” Available: https://sei.cmu.edu/library/quality-attribute-workshop-collection/

Contribution Table

Member name and ID	Actual contribution
[Ahmed Adel / 221004576]	Everything